

Orientación a Objetos II

2026

Explicación de práctica
Semana del 18 de mayo



FACULTAD DE INFORMÁTICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Ejercicios de la semana

En el cuadernillo de Frameworks

- Ejercicio 1: SingleThreadTCPFramework
 - Ejercicio i - Extensión del framework
 - Ejercicio ii - PasswordServer
 - Ejercicio iii - Repeat server
- Ejercicio 2: Java Logging
- Ejercicio 3: Extensión de Frameworks
- Ejercicio 4: tcp.server.reply

Frameworks - SingleThreadTCPFramework

- Leer el material
- En los documentos se explican conceptos teóricos, cómo funciona y cómo se usa
SingleThreadTCPFramework

Material Framework Caja Blanca

Editar



Frameworks I.pdf



OO2_SingleThreadEchoServer.pdf



SingleThreadTCPServer.zip

Para ejecutar el código, en una terminal / consola:

1. Compilar:

```
javac EchoServer.java VoidServer.java SingleThreadTCPServer.java
```

2. Ejecutar la **clase concreta** que queremos instanciar, indicando también un **número de puerto**:

```
java VoidServer 8888
```

```
java EchoServer 1234
```

3. Conectarse con algún cliente para enviar un mensaje y recibir la respuesta

- **Linux:** se puede usar Netcat (commando **nc**)
- **Windows:**
 - **Windows Subsystem for Linux (WSL)**, usar jdk y luego usar Netcat
 - **Telnet:** se usa por consola de manera similar. Requiere habilitarlo en "características de Windows"
 - **PuTTY:** Es otro cliente Telnet

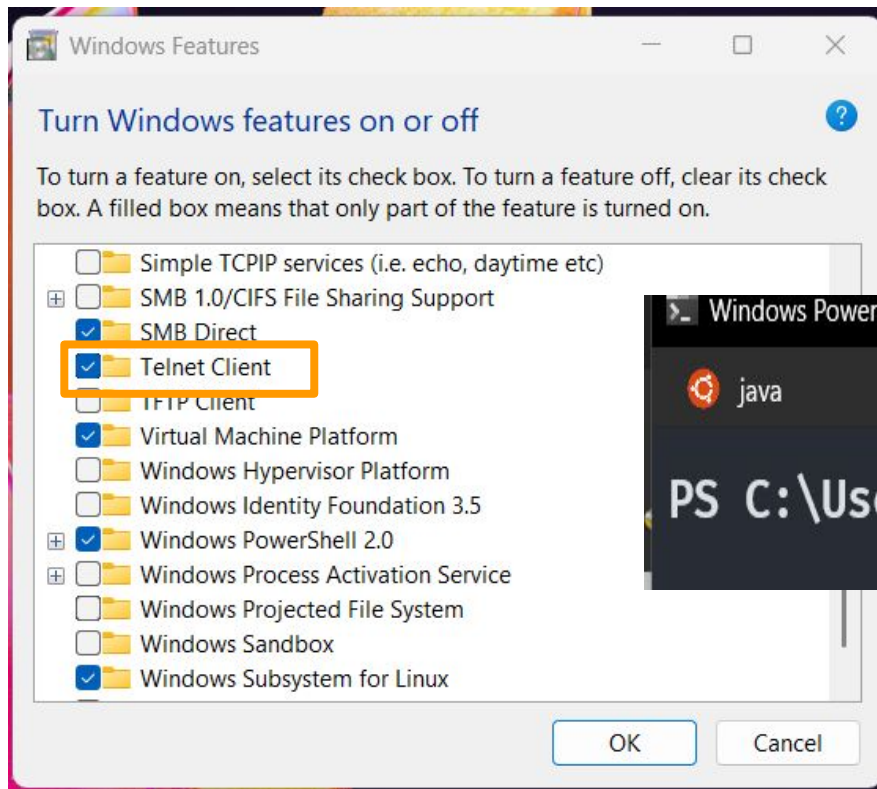
Ejecución - Linux / Windows con WSL

```
nc
nc
aortu .../SingleThreadTCPServer ? master ? v21.0.
2 16:54 ls
DayTimeServer.java SingleThreadTCPServer.java
EchoServer.java VoidServer.java
aortu .../SingleThreadTCPServer ? master ? v21.0.
2 16:54 javac EchoServer.java
aortu .../SingleThreadTCPServer ? master ? v21.0.
2 16:54 java EchoServer 1234
EchoServer server listening on port: 1234
Client connected from: 127.0.0.1:60962
Received message: esto es una prueba from 127.0.0.1:60962
aortu ~ 16:54 nc localhost 1234
esto es una prueba
esto es una prueba
```

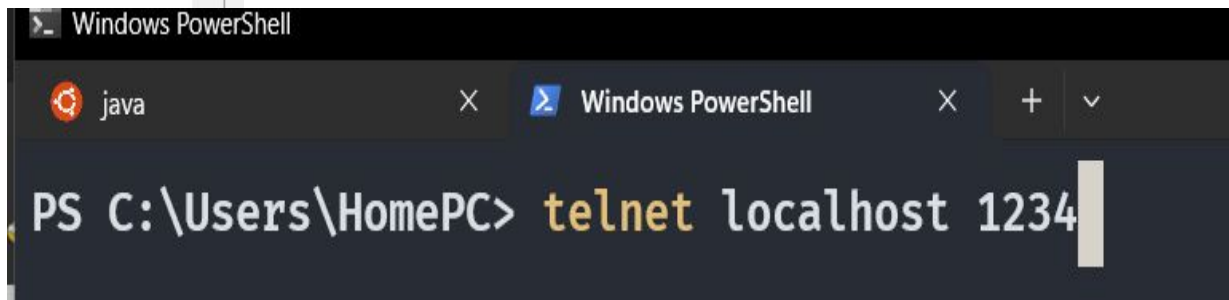
Tipear mensaje + enter

Respuesta de "echo"

Ejecución - Windows (Telnet)



Panel de control > Programas >
Activar o desactivar características
de Windows

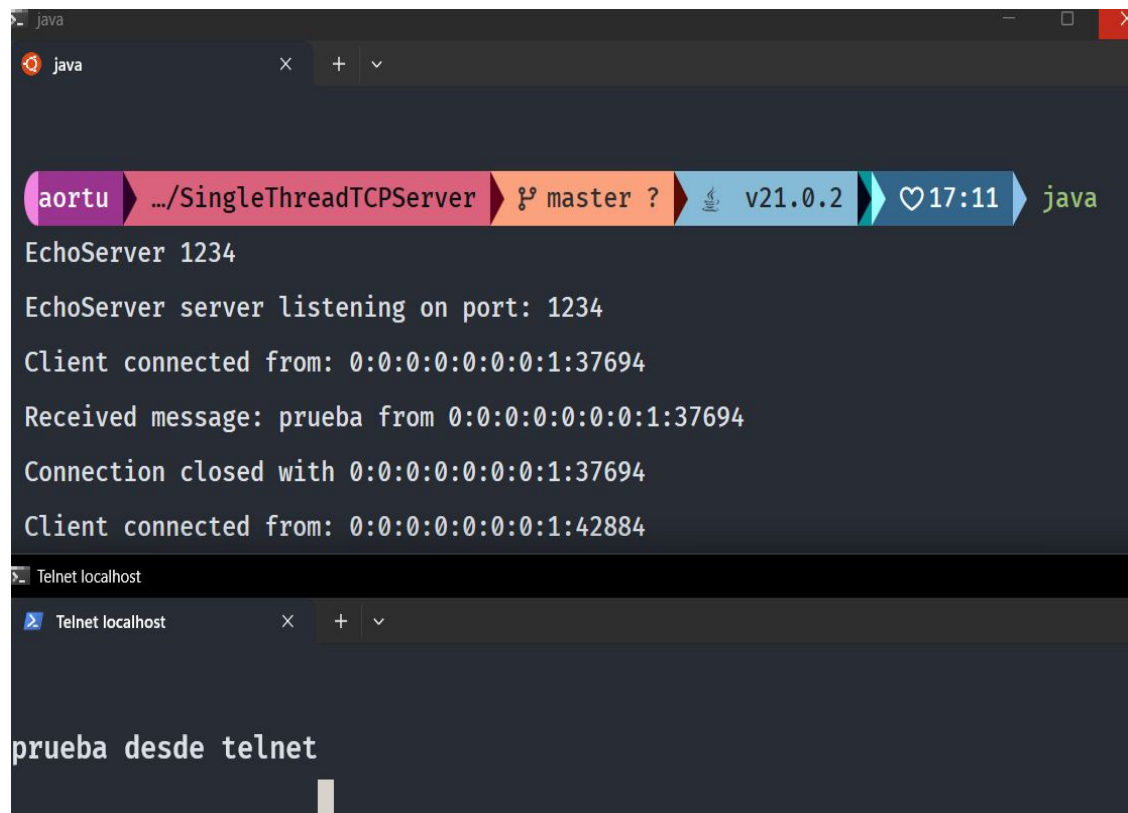


Ejecución - Windows (Telnet)

Abrir un Powershell y escribir

`telnet localhost puerto`

Luego tipear mensaje + enter



The screenshot shows two windows. The top window is titled 'java' and contains the following text: 'aortu .../SingleThreadTCPServer master ? v21.0.2 17:11 java', 'EchoServer 1234', 'EchoServer server listening on port: 1234', 'Client connected from: 0:0:0:0:0:0:0:1:37694', 'Received message: prueba from 0:0:0:0:0:0:0:1:37694', 'Connection closed with 0:0:0:0:0:0:0:1:37694', and 'Client connected from: 0:0:0:0:0:0:0:1:42884'. The bottom window is titled 'Telnet localhost' and contains the text 'prueba desde telnet'.

```
java
aortu .../SingleThreadTCPServer master ? v21.0.2 17:11 java
EchoServer 1234
EchoServer server listening on port: 1234
Client connected from: 0:0:0:0:0:0:0:1:37694
Received message: prueba from 0:0:0:0:0:0:0:1:37694
Connection closed with 0:0:0:0:0:0:0:1:37694
Client connected from: 0:0:0:0:0:0:0:1:42884

Telnet localhost
prueba desde telnet
```

Frameworks - Ejercicio 1

Refactorizar el método

SingleThreadTCPFramework::handleClient(Socket) para convertirlo en un Template Method.

- Este método debe incluir métodos hook opcionales, es decir, métodos hooks que pueden ser implementados por las subclases o no.
- ¿Qué cosas creemos que podrían variar cuando un programador utiliza este framework y quiere construir una aplicación?

```
1 private final void handleClient(Socket clientSocket) {
2     try (
3         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
4         BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {
5         String inputLine;
6         while ((inputLine = in.readLine()) != null) {
7             System.out.println("Received message: " + inputLine + " from "
8                 + clientSocket.getInetAddress().getHostAddress() + ":" + clientSocket.getPort());
9
10            if ("".equalsIgnoreCase(inputLine)) {
11                break; // Client requested to close the connection
12            }
13            this.handleMessage(inputLine, out);
14        }
15        System.out.println("Connection closed with " + clientSocket.getInetAddress().getHostAddress() + ":"
16            + clientSocket.getPort());
17    } catch (IOException e) {
18        System.err.println("Problem with communication with client: " + e.getMessage());
19    } finally {
20        try {
21            clientSocket.close();
22        } catch (IOException e) {
23            System.err.println("Error closing socket: " + e.getMessage());
24        }
25    }
26 }
```

```
1 private final void handleClient(Socket clientSocket) {
2     try (
3         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
4         BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {
5         String inputLine;
6         while ((inputLine = in.readLine()) != null) {
7             System.out.println("Received message: " + inputLine + " from "
8                 + clientSocket.getInetAddress().getHostAddress() + ":" + clientSocket.getPort());
9
10            if ("".equalsIgnoreCase(inputLine)) {
11                break; // Client requested to close the connection
12            }
13            this.handleMessage(inputLine, out);
14        }
15        System.out.println("Connection closed with " + clientSocket.getInetAddress().getHostAddress() + ":"
16            + clientSocket.getPort());
17    } catch (IOException e) {
18        System.err.println("Problem with communication with client: " + e.getMessage());
19    } finally {
20        try {
21            clientSocket.close();
22        } catch (IOException e) {
23            System.err.println("Error closing socket: " + e.getMessage());
24        }
25    }
26 }
```

```
1 private final void handleClient(Socket clientSocket) {
2     try (
3         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
4         BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {
5         String inputLine;
6         while ((inputLine = in.readLine()) != null) {
7             this.beforeCommunication(clientSocket, inputLine);
8
9             if ("".equalsIgnoreCase(inputLine)) {
10                 break; // Client requested to close the connection
11             }
12             this.handleMessage(inputLine, out);
13         }
14         System.out.println("Connection closed with " + clientSocket.getInetAddress().getHostAddress() + ":" + clientSocket.getPort());
15     }
```

```
1 protected void beforeCommunication(Socket clientSocket, String inputLine) {
2     System.out.println("Received message: " +
3         inputLine + " from " +
4         clientSocket.getInetAddress().getHostAddress() +
5         ":" + clientSocket.getPort());
6 }
```

```
24 }
25 }
```

```
1 protected void beforeCommunication(Socket clientSocket, String inputLine) {  
2     System.out.println("Received message: " +  
3         inputLine + " from " +  
4         clientSocket.getInetAddress().getHostAddress() +  
5         ":" + clientSocket.getPort());  
6 }
```

¿Que podría hacer un programador al instanciar el framework y redefinir éste método? Sólo algunas ideas...

- **Auditoría:** guardar en un log / base de datos quién accedió, desde que IP, en qué fecha/hora ...
- **Seguridad:** implementar alguna política para que ciertos accesos no sean permitidos

```

1 private final void handleClient(Socket clientSocket) {
2     try (
3         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
4         BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {
5         String inputLine;
6         while ((inputLine = in.readLine()) != null) {
7             this.beforeCommunication(clientSocket, inputLine);
8
9             if ("".equalsIgnoreCase(inputLine)) {
10                 break; // Client requested to close the connection
11             }
12             this.handleMessage(inputLine, out);
13         }
14         System.out.println("Connection closed with " + clientSocket.getInetAddress()
15             + clientSocket.getPort());
16     } catch (IOException e) {
17         System.err.println("Problem with communication with client: " + e.getMessage());
18     } finally {
19         try {
20             clientSocket.close();
21         } catch (IOException e) {
22             System.err.println("Error closing socket: " + e.getMessage());
23         }
24     }
25 }

```

- ¿Qué otras cosas creemos que podrían variar cuando un programador utiliza este framework y quiere construir una aplicación?

Ejercicio 1: SingleThreadTCPFramework

Extienda el framework para permitir que la “palabra” que produce el cierre de la sesión con un cliente sea configurable. Evalúe las siguientes cuatro alternativas e implemente la que considere más adecuada:

- Una **variable en SingleThreadTCPServer** que se configura desde el método main() de las subclases.
- Un **método (hook)** que retorna un booleano resultado de evaluar la condición.
- Un **método (hook)** que retorna un String que es la palabra de término de sesión.
- Una **jerarquía de Strategies** que implementan cada una de las condiciones de cierre de sesión.

Para ayudarlo en definir cuál sería la alternativa que considera más apropiada, puede usar los siguientes aspectos (pero no limitarse a):

- Esfuerzo de implementación dentro del framework
- Facilidad de uso para los programadores usuarios del framework
- Limitaciones de la solución; es decir, que tan flexible es la alternativa elegida

```
1 private final void handleClient(Socket clientSocket) {
2     try (
3         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
4         BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {
5         String inputLine;
6         while ((inputLine = in.readLine()) != null) {
7             System.out.println("Received message: " + inputLine + " from "
8                 + clientSocket.getInetAddress().getHostAddress() + ":" + clientSocket.getPort());
9
10            if ("".equalsIgnoreCase(inputLine)) {
11                break; // Client requested to close the connection
12            }
13            this.handleMessage(inputLine, out);
14        }
15        System.out.println("Connection closed with " + clientSocket.getInetAddress().getHostAddress() + ":"
16            + clientSocket.getPort());
17    } catch (IOException e) {
18        System.err.println("Problem with communication with client: " + e.getMessage());
19    } finally {
20        try {
21            clientSocket.close();
22        } catch (IOException e) {
23            System.err.println("Error closing socket: " + e.getMessage());
24        }
25    }
26 }
```

Ejercicio 1: SingleThreadTCPFramework

Implemente un servidor **PasswordServer**. Este servidor debe generar una password a partir de los tres argumentos que recibe en el mensaje enviado por el cliente.

- Arg[0]: cadena de caracteres (letras) permitidas para utilizar en la password
- Arg[1]: cadena de caracteres (números de 0 a 9) permitidos para utilizar en la password
- Arg[2]: cadena de caracteres especiales permitidos para utilizar en la password

Las passwords deben ser generadas de forma aleatoria y cumplir con las siguientes reglas:

- Tener una longitud de 8 caracteres
- Contener letras, al menos un número y un solo carácter especial

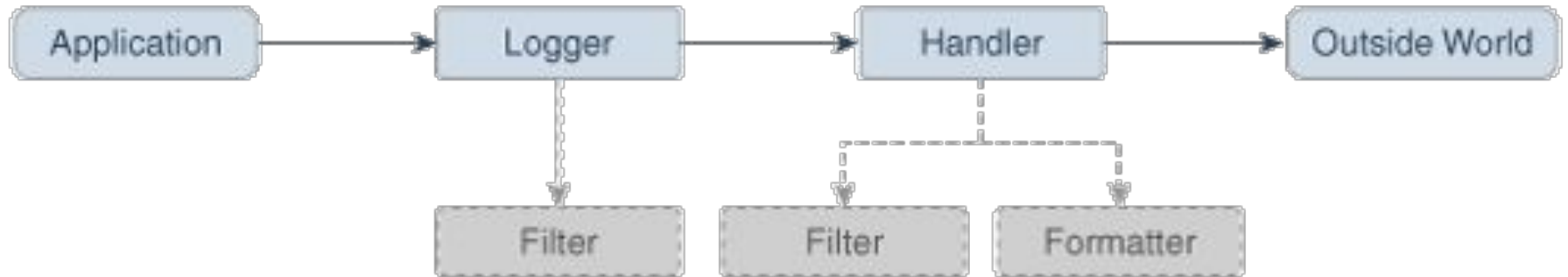
Ejercicio 1: SingleThreadTCPFramework

Implemente un servidor **RepeatServer**. Este servidor debe repetir un string a partir de los argumentos que recibe en el mensaje enviado por el cliente:

- Arg[0]: es el string a repetir. Este argumento es requerido y no puede ser nulo o vacío.
- Arg[1]: es la cantidad de veces que debe repetir el string. Este argumento es requerido y debe ser un número entero mayor a 0.
- Arg[2]: es un carácter que se utiliza para delimitar los strings repetidos. Este argumento es opcional, y en caso de que el cliente no lo especifique, es un espacio en blanco.

Ejercicio 2: Java Logging

En las clases teóricas de frameworks se trabajó con el framework de [Logging de Java](#). Con lo visto en teoría y leyendo la documentación provista en el link anterior, resuelva los siguientes ejercicios.



Ejercicio 2: Java Logging

Parte A: En este apartado utilizaremos el framework como usuarios, aprovechando las implementaciones ya provistas por éste.

i) Tomando su implementación del ejercicio de protección para el acceso a una base de datos (Cuadernillo patrones, ejercicio 20), incorpore logging de mensajes en las siguientes situaciones:

- Acceso válido para búsquedas a la base de datos con nivel INFO.
- Acceso válido para inserciones a la base de datos con nivel WARNING.

```
2020-11-11 13:52:12 INFO app - XYZ Integration API Manage
2020-11-11 13:52:15 INFO app - Loading configurations..
2020-11-11 13:52:18 INFO app - *** InstanceID API V2_I02
2020-11-11 13:52:18 INFO app - *** BaseURL http://10.244.
2020-11-11 13:52:19 INFO app - *** LogLevel 05 (DEBUG)
2020-11-11 13:52:12 DEBUG App:22 - Initializing Swagger U
2020-11-11 13:52:14 DEBUG App:28 - Generating schemata..
2020-11-11 13:52:14 DEBUG App:30 - Initializing REST serv
2020-11-11 13:52:15 DEBUG App:32 - Generating documentati
2020-11-11 13:52:18 DEBUG App:36 - Loading adapters..
2020-11-11 13:52:21 DEBUG Adapters:10 - Loading adapter d
2020-11-11 13:52:22 DEBUG Adapters:16 - Loading adapter m
2020-11-11 13:52:26 DEBUG Docs:38 - document-store adapte
2020-11-11 13:52:27 DEBUG Mongo:38 - mongo adapter initia
2020-11-11 13:52:28 DEBUG Adapters:38 - Adapter: docs/v1
2020-11-11 13:52:28 DEBUG Adapters:38 - Adapter: mongo/v1
2020-11-11 13:52:31 INFO app - Started listening...
2020-11-11 13:52:27 ERROR Oracle:22 - Failed to write to
2020-11-11 13:52:27 ERROR Users:24 - Failed to create JSO
2020-11-11 13:52:14 DEBUG BatchWriter:28 - Successfully c
2020-11-11 13:52:20 DEBUG BatchWriter:35 - Successfully i
```

Ejercicio 2: Java Logging

Parte B:

i) Extienda el framework de logging de Java para poder formatear los mensajes de log de las siguientes maneras:

- Realizar el registro de un mensaje completamente en mayúscula, similar al ejercicio resuelto en teoría. Utilice este formato para realizar instancias similares a las realizadas en el inciso A.
- Realizar el registro de un mensaje en formato JSON. El resultado de hacer logging con este formato debería ser un String en formato JSON con los campos *message* y *level* y como valores el string registrado y el nivel de severidad. Utilice este formato para realizar instancias similares a las realizadas en el inciso A.

Por ejemplo:

```
logger.info("Logging with json");
```

Debería generar como salida el siguiente mensaje:

```
{ "message": "Logging with json", "level": "info" }
```



Ejercicio 2: Java Logging

ii) Realice las siguientes dos extensiones al framework:

- Agregar un Handler que aplique un filtro de palabras a ocultar antes de ejecutar un Handler existente. El mismo debe poder configurarse con una lista de palabras a ocultar y reemplazar cada aparición de alguna de éstas con el String "***".

Por ejemplo, si se configura este handler con la lista ["switch-statements"] debería suceder que:

```
logger.info("I love switch-statements");
```

en realidad realice el log del String:

```
"I love ***"
```



Frameworks - Logging

ii) Realice las siguientes dos extensiones al framework:

- Mediante un handler posibilite la opción de enviar por correo electrónico los mensajes registrados por el framework. Al final del documento encontrará un anexo con una solución general al envío de mails desde una aplicación java. Si lo considera conveniente, puede seguir estos pasos para resolver el envío de mails, adaptando lo que considere necesario para que la funcionalidad se ejecute al momento de realizar el logging de un mensaje.
- Aplique ambos handlers en los escenarios de logging planteados en el inciso de la parte A.

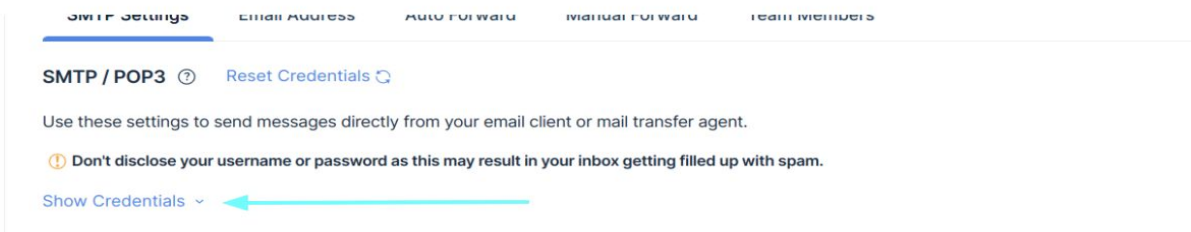


Frameworks - Anexo Mailtrap

Anexo Mailtrap

Mailtrap es una herramienta que implementa un “falso servidor SMTP”. Es ideal para pruebas, ya que nos permite recibir correos electrónicos en una bandeja de entrada en la nube. Al registrarnos, nos permite crear tantas inbox cómo querramos, y nos provee de las credenciales para poder enviar los mails desde nuestras aplicaciones:

1. Ingrese a <https://mailtrap.io/> y regístrese.
2. En el dashboard, presione el botón “Add project”. Nos va a pedir que indiquemos un nombre para crearlo.
3. Una vez creado el proyecto, presione el botón “Add inbox”, en donde nuevamente nos pedirá indicar el nombre.
4. En la tabla aparecerá la bandeja de entrada creada en el paso anterior, haga click sobre su nombre (o bien, en el botón “settings”).
5. Se nos presentan los datos del inbox; necesitamos obtener la configuración de conexión para poder incorporarla en nuestro proyecto Java. Para ver estas credenciales, haga click en “Show credentials”.

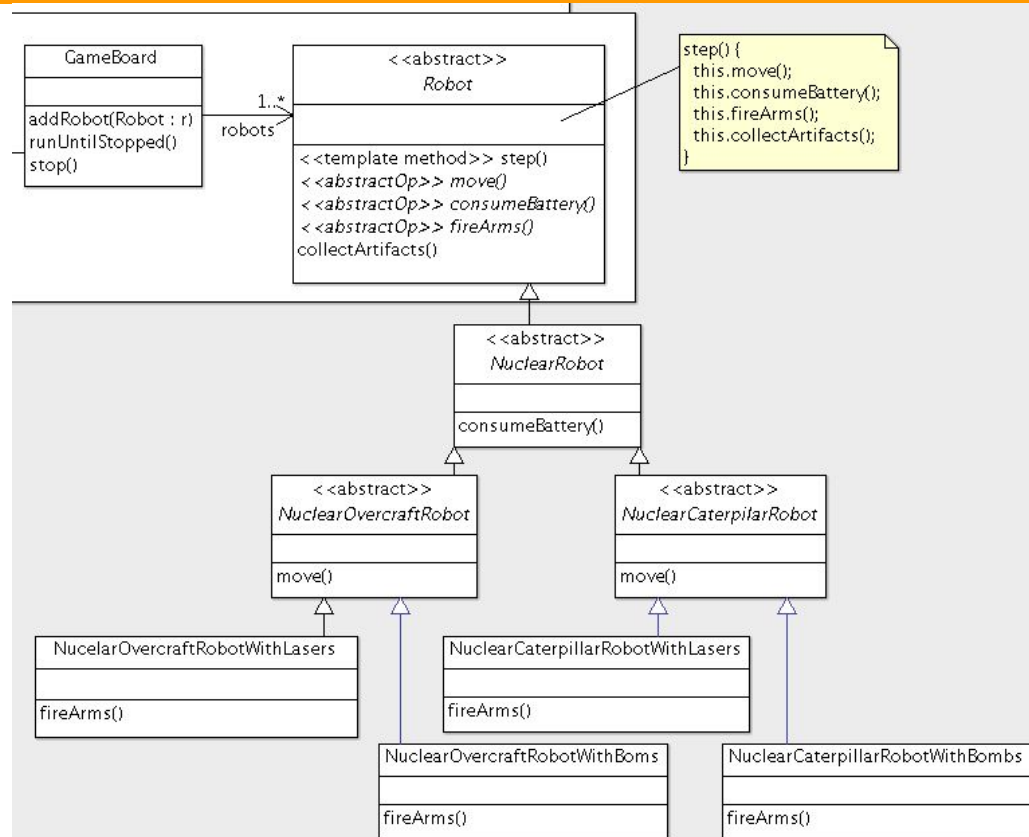


Ejercicio 3: Extensión de Frameworks

1) Respecto a la sección **Plantillas y herencia** del recurso de aprendizaje

- Responda las siguientes preguntas:
 - a. ¿Qué debo hacer si aparece una nueva fuente de energía (por ejemplo, paneles solares con baterías)? ¿Cuántas y cuáles clases debo agregar en caso de querer todas las variantes de robots posibles para este nuevo tipo de fuente de energía?
 - b. ¿Puedo cambiarle, a un robot existente, el sistema de armas sin tener que instanciar el robot de nuevo?
 - c. ¿Dónde almacenaría usted el nivel de carga de la batería? ¿Qué implicaría eso si antes de disparar el láser hay que garantizar que la fuente de energía puede satisfacer el consumo del arma?
- Implemente en Java lo necesario para satisfacer el punto a. Luego, agregue un nuevo ejemplo de uso del framework instanciando uno de los robots con la nueva fuente de energía.

Ejercicio 3: Extensión de Frameworks

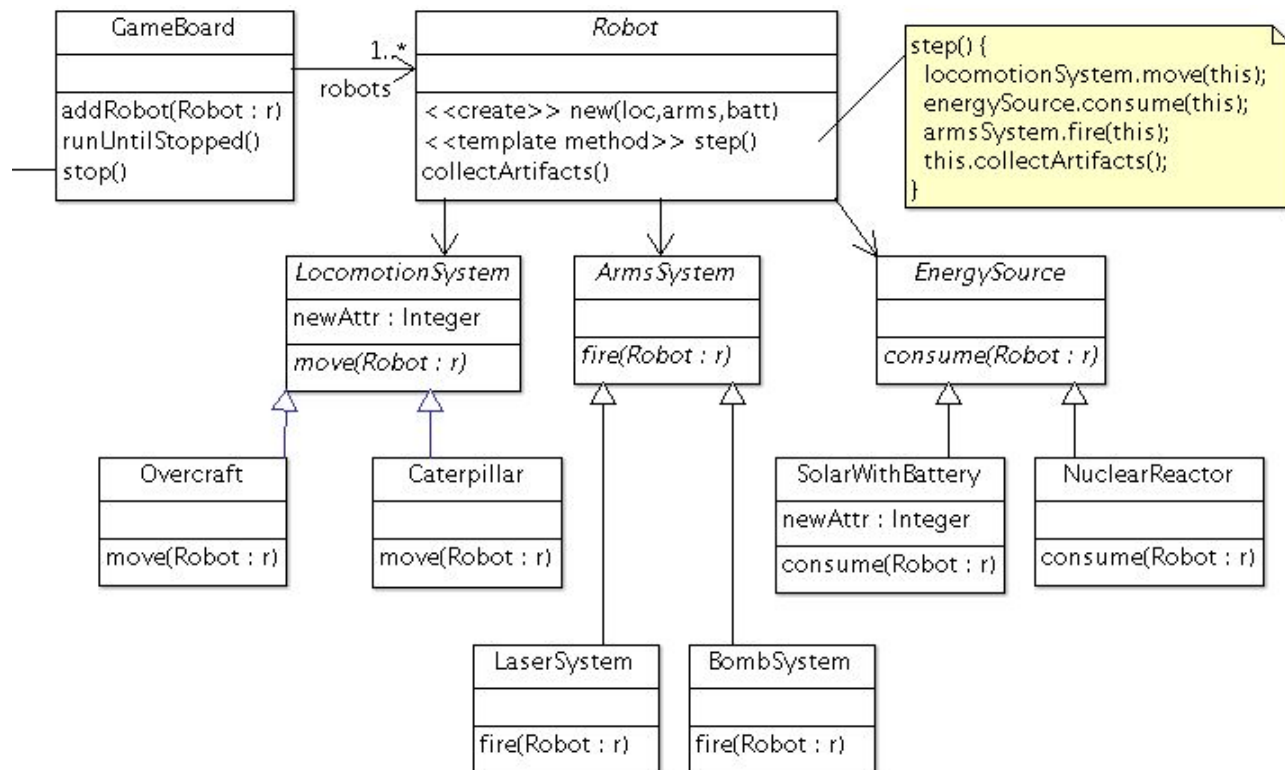


Ejercicio 3: Extensión de Frameworks

2) Respecto a la sección **Plantillas y composición** del recurso de aprendizaje

- Responda las siguientes preguntas:
 - a. ¿Qué debo hacer si aparece una nueva fuente de locomoción (por ejemplo, motor con ruedas con tracción 4x4)? ¿Cuántas y cuáles clases debo agregar en caso de querer todas las variantes de robots posibles para este nuevo tipo de sistema de locomoción?
 - b. ¿Puedo cambiarle, a un robot existente, el sistema de armas sin tener que instanciar el robot de nuevo?
 - c. ¿Dónde almacenaría usted el nivel de carga de la batería? ¿Qué implicaría eso si antes de disparar el láser hay que garantizar que la fuente de energía puede satisfacer el consumo del arma?
- Implemente en Java lo necesario para satisfacer el punto a. Luego, agregue un nuevo ejemplo de uso del framework instanciando uno de los robots con la nueva forma de locomoción.

Ejercicio 3: Extensión de Frameworks



Ejercicio 4: tcp.server.reply

A partir del código compartido en teoría de tcp.server.reply.

i) Reimplementar el servidor PasswordServer empleando un enfoque basado en composición de objetos utilizando los componentes del framework tcp.server.reply vistos en teoría

Recordando el ejercicio 1...

El servidor debe generar una password a partir de los tres argumentos que recibe en el mensaje enviado por el cliente.

- Arg[0]: cadena de caracteres (letras) permitidas para utilizar en la password
- Arg[1]: cadena de caracteres (números de 0 a 9) permitidos para utilizar en la password
- Arg[2]: cadena de caracteres especiales permitidos para utilizar en la password

Las passwords deben ser generadas de forma aleatoria y cumplir con las siguientes reglas:

Tener una longitud de 8 caracteres

Contener letras, al menos un número y un solo carácter especial

Dominio: TCP Servers

SingleThreadTCPServer (whitebox)

Cookbook:

1. Subclasificar SimpleThreadTCPServer
 - a. Debe implementar Main(String[])
 - i. crear una instancia
 - ii. enviar método startLoop(String[])
 - b. Debe implementar handleMessage(String)
⇒ hook

tcp.server.reply (blackbox)

Cookbook

1. En un objeto “contexto”
 - a. instanciar un MessageHandler
 - i. Echo,
 - ii. Void
 - b. Instanciar ConnectionHandler con el MessageHandler
 - i. SimpleConnectionHandler
 - ii. MultiConnectionHandler
 - c. Instanciar TCPControlLoop con ConnectionHandler
 - d. Enviar método startLoop() al TCPControlLoop

Ejercicio 4: tcp.server.reply

```
public class TCPControlLoop {  
  
    private IConnectionHandler connectionHandler;  
    public TCPControlLoop() {  
        this.connectionHandler = new SingleConnectionHandler();  
    }  
  
    public TCPControlLoop(IConnectionHandler connHandler) {  
        this.connectionHandler = connHandler;  
    }  
  
    public final void startLoop(String[] args) {  
        checkArguments(args);  
  
        int portNumber = Integer.parseInt(args[0]);  
  
        try (ServerSocket serverSocket = new ServerSocket(portNumber)) {  
            displaySocketInformation(portNumber);  
            while (true) {  
                connectionHandler.handleConnection(serverSocket);  
            }  
        } catch (IOException e) {  
            displayAndExit(portNumber);  
        }  
    }  
}
```

Ejercicio 4: tcp.server.reply

```
public interface IConnectionHandler {  
    void handleConnection(ServerSocket serverSocket) throws IOException;  
}
```

```
public interface IMessageHandler {  
    void handleMessage(String message, PrintWriter out);  
}
```

Ejercicio 4: tcp.server.reply

```
public class SingleConnectionHandler implements IConnectionHandler {
    private IMessageHandler messageHandler;

    public SingleConnectionHandler(IMessageHandler messageHandler, /*EndSessionPolicy endSessionPolicy, */
        ITCPSession sessionHandler) {
        this.messageHandler = messageHandler;
    }

    public SingleConnectionHandler(IMessageHandler messageHandler /*, EndSessionPolicy endSessionPolicy */) {
        this.messageHandler = messageHandler;
    }
}
```

Ejercicio 4: tcp.server.reply

```
public class EchoHandler implements IMessageHandler {  
    public void handleMessage(String message, PrintWriter out) {  
        out.println(message);  
    }  
}
```

```
public class EchoApp {  
    Run | Debug  
    public static void main(String[] args) {  
        new TCPControlLoop(new SingleConnectionHandler(new EchoHandler())).startLoop(args);  
    }  
}
```

Ejercicio 4: tcp.server.reply

ii) Modifique el framework para que la condición de cierre de una conexión sea configurable con strings provistos por las instanciaciones.

Recordando el ejercicio 1...

- a. Una variable en SingleThreadTCPServer que se configura desde el método main() de las subclases.
- b. Un método (hook) que retorna un booleano resultado de evaluar la condición.
- c. Un método (hook) que retorna un String que es la palabra de término de sesión.
- d. Una jerarquía de Strategies que implementan cada una de las condiciones de cierre de sesión.

Ejercicio 4: tcp.server.reply

iii) Respecto a las dos formas vistas para implementar los servidores (PasswordServer ejercicio 1)

iii) y 4) i):

- ¿Qué debe hacer un desarrollador para extender el framework en cada una de las formas? Especifique qué clases debe subclasificar o implementar, qué métodos debe definir.
- ¿Cuánto conocimiento necesita tener el desarrollador sobre la estructura interna del framework para instanciarlo? ¿Y para extenderlo?
- ¿Qué técnica usarías si tuviera que ofrecer muchas configuraciones posibles para el servidor? ¿Por qué?
- Identifique los hotspots y frozen spots en cada una de las implementaciones.
- Considerando las dos formas de implementación del servidor PasswordServer, los programadores pueden asegurar que hay inversión de control? Justifique su respuesta identificando en qué parte se produce la inversión de control en cada uno de los casos.

Examen Parcial

- **Primera fecha:** **Sábado 6 de junio, 13,00 horas**
aulas 4, 5, 9, 10A y 11
- **1er Recuperatorio:** **Sábado 27 de junio, 13,00 horas**
aulas 4, 5, 9, 10A y 11
- **2do Recuperatorio:** **Sábado 11 de julio, 13,00 horas**
aulas 4, 5, 9, 10A y 11
+ promoción

Atención - Para tener en cuenta...



Para Patrones:

- Es importante conocer los roles de cada clase dentro del patrón
 - Si nos preguntan algo del “context” del State tenemos que saber que objeto cumple ese rol con ese nombre

Para Refactoring

- Es importante reconocer los bad smells y los refactorings por sus nombres (en inglés)
 - Por ej. nos pueden decir “aplique extract method en las líneas 3-7”
 - ¿Existe feature envy en la línea 8?

Guías y material de referencia



Catálogo de Bad Smells de Objetos 2

Este archivo contiene un catálogo con la clasificación y

